

The Agent Supervisor Pattern with LangGraph

The **Agent Supervisor Pattern** is a common architecture for building multi-agent systems.

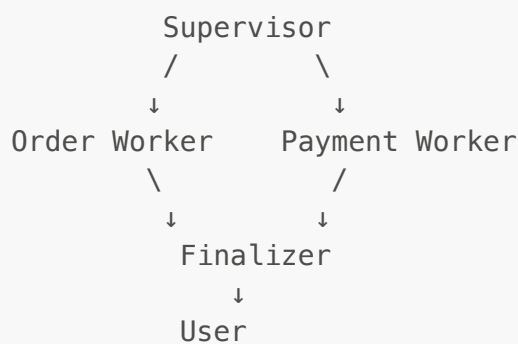
Instead of allowing agents to directly control the entire workflow, a **Supervisor Agent** coordinates a group of specialized worker agents.

For example, an order-processing system might contain:

- **Order Worker** — handles order-related requests.
- **Payment Worker** — handles payment-related requests.
- **Supervisor** — decides which worker should handle a request and coordinates the overall workflow.

1. Basic Architecture

The architecture can be represented as:



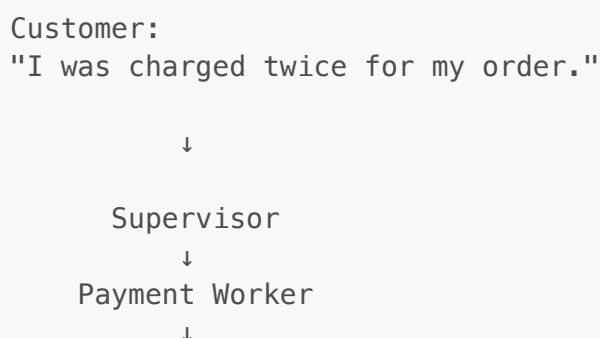
The Supervisor is responsible for orchestration.

2. Supervisor Responsibilities

The Supervisor typically performs two important tasks:

1. **Route the request to an appropriate worker**
2. **Coordinate the final response**

For example:



```
Payment Result
  ↓
Finalizer
  ↓
Customer Response
```

The Supervisor determines that the request is related to payment and selects the Payment Worker.

3. Specialized Workers

Workers should generally have focused responsibilities.

For example:

```
Order Worker
  → order status
  → delivery information

Payment Worker
  → payment status
  → payment failures
  → duplicate charges
```

This separation prevents one agent from having to handle every type of task.

Each worker can have its own prompt, tools, and domain-specific logic.

4. Routing

The Supervisor can use an LLM to determine the appropriate worker.

Conceptually:

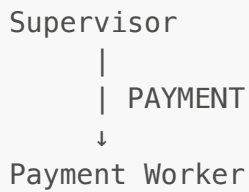
```
result = llm.invoke("""
Choose the appropriate worker:

ORDER
PAYMENT
""")
```

The Supervisor might produce:

```
PAYMENT
```

LangGraph can then route execution to the corresponding node.



The important point is that the **Supervisor controls the routing**, rather than the workers deciding which agent should execute next.

5. Shared State

The Supervisor and workers can communicate through LangGraph state.

For example:

```

class OrderState(TypedDict):
    order_id: str
    request: str
    worker: str
    worker_result: str
    final_response: str
  
```

The Supervisor writes the selected worker:

```
worker = "payment"
```

The Payment Worker then writes its result:

```
worker_result = "Payment is being investigated."
```

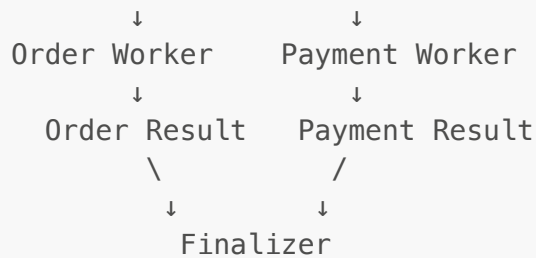
The finalizer can use that result to construct the customer-facing response.

6. Aggregating Results

A Supervisor architecture becomes especially useful when multiple workers are involved.

For example:





The finalizer can combine the results into a single response.

For example:

```
Order status: Shipped
Payment status: Successful
```

↓

```
"Your order has shipped and the payment
was successfully processed."
```

This is the **aggregation** part of the pattern.

7. Finalizing the Reply

After a worker completes its task, the finalizer can use an LLM to produce the user-facing response.

Conceptually:

```
result = llm.invoke(f"""
Customer request:
{request}

Specialist result:
{worker_result}

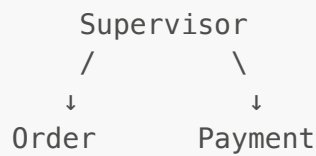
Create the final response.
""")
```

The final response should normally hide internal implementation details such as worker names or routing decisions.

8. Supervisor vs. Peer-to-Peer

The Supervisor Pattern differs from peer-to-peer agent collaboration.

Supervisor



The Supervisor controls the workflow.

Peer-to-Peer



Agents can directly hand work to one another.

The choice depends on the workflow.

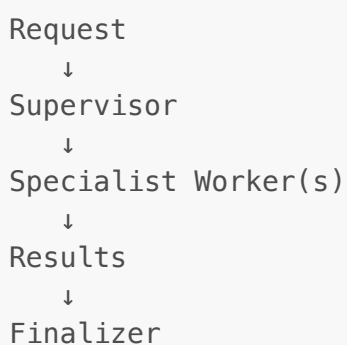
A Supervisor provides a clear central control point, while peer-to-peer interaction provides more decentralized collaboration.

9. Why Use the Supervisor Pattern?

The pattern is useful when:

- Different tasks require specialized agents.
- A central component should control routing.
- Worker responsibilities should remain focused.
- Results from multiple workers need to be combined.
- The final response should be generated consistently.

It also makes the overall workflow easier to visualize:



↓
User

Key Takeaway

The Agent Supervisor Pattern separates **coordination** from **specialized work**.

```
Supervisor
  ↓
Decides
  ↓
Delegates
  ↓
Collects results
  ↓
Finalizes
```

In LangGraph, the Supervisor, workers, and finalizer can be represented as graph nodes, while the graph state provides a common mechanism for passing information between them.

The core idea is simple:

The Supervisor decides who should work; specialized agents perform the work; the results are combined into the final response.